



ค่ายโอลิมปิกวิชาการ วิชาคอมพิวเตอร์

Greedy algorithm

ปรับปรุงจาก เอกสารของ ดร.ณัฐกิตติ์ จิตรเอื้อตระกูล

ผศ.ดร.เจียบวุฒิ รัตนวิไลสกุล chaibwoot.r@sci.kmutnb.ac.th

ภาควิชาวิทยาการคอมพิวเตอร์และสารสนเทศ

มหาวิทยาลัยเทคโนโลยีพระจอมเกล้าพระนครเหนือ

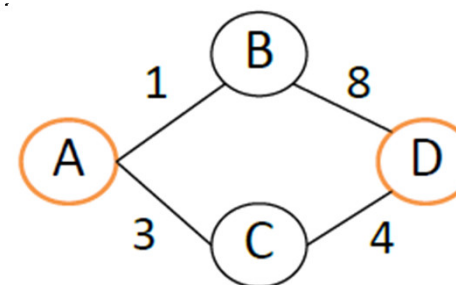


อัลกอริทึมเชิงละโมภ (Greedy algorithm)

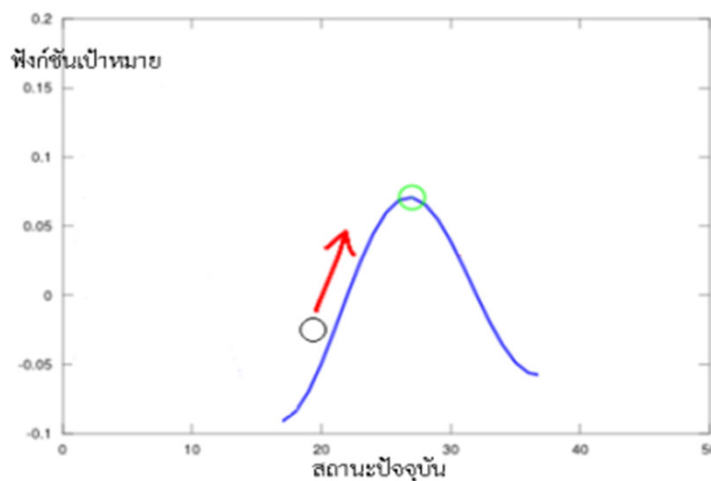
- อัลกอริทึมเชิงละโมภเป็นเทคนิคที่เหมาะสมในการ**แก้ปัญหาเพื่อค้นหาคำตอบที่เหมาะสมที่สุด** (optimal solution)
- หลักการเชิงละโมภ ก็คือการลดเวลาโดยการตัดสินใจเลือกคำตอบย่อย (sub solutions) ที่ดีที่สุดในขณะนั้น โดยที่คำตอบที่เลือกจะต้องไม่ขัดแย้งกับเงื่อนไขบังคับของปัญหา (problem constraint) โดยคาดหวังว่าคำตอบที่เลือกจะได้คำตอบที่เหมาะสมที่สุด
- ดังนั้นประสิทธิภาพของ Greedy จึงจะขึ้นอยู่กับ เกณฑ์ที่**นำไปใช้ในการตัดสินใจ** (greedy criterion) ในการ**เลือกแต่ละคำตอบย่อย**

Greedy Algorithm

- ข้อดีของอัลกอริทึมเชิงละโมภ
 - การพัฒนาโปรแกรมทำได้ง่าย
 - อัลกอริทึม greedy มักใช้เวลาในการทำงานที่ดี
บ่อยครั้งเป็นฟังก์ชัน polynomial
- ข้อเสีย
 - ไม่รับประกันว่าคำตอบที่ได้จะเป็นคำตอบที่ดีที่สุด (optimal solution) เสมอ เนื่องจากการเลือกคำตอบย่อยที่ดีที่สุดทุกครั้งอาจไม่ได้นำไปสู่คำตอบที่ดีที่สุดเสมอไป

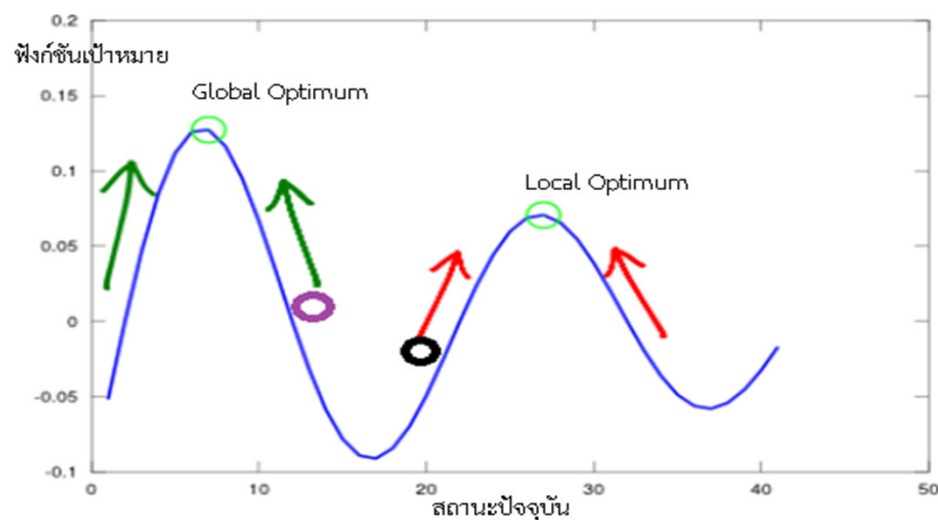


Greedy Algorithm



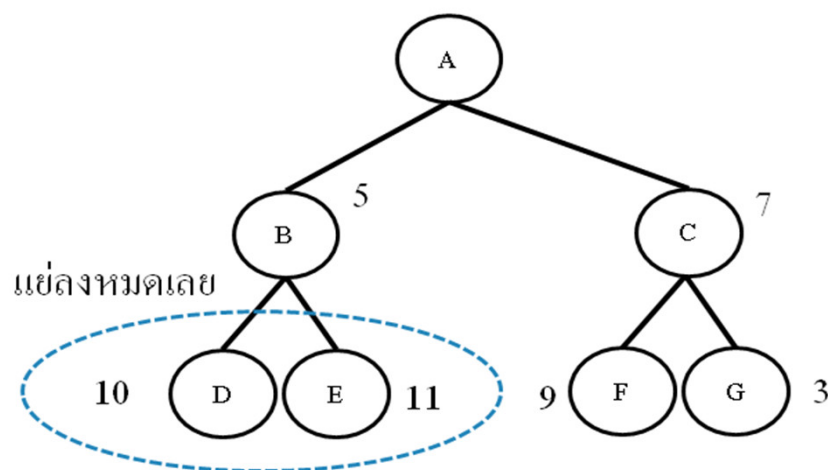
หลักการทำงานของเทคนิคเชิงละโมภจะเลือกสิ่งที่ดีที่สุดที่**ตัวค้นหา (Agent)** นั้นพบ จากรูป ต้องการหาจุดสูงสุด ณ ตำแหน่ง วงกลมสีแดงหรือตัวค้นหามีทางเลือกสองทาง คือขึ้นไปข้างบน กลับลงมาข้างล่าง ณ ตำแหน่งนี้ตัวค้นหาจะเห็นเส้นทางขึ้นและเส้นทางลง จากหลักการทำงานของเทคนิคเชิงละโมภ ต้องขึ้นไปข้างบน เพราะพิจารณาแล้ว ว่าขึ้นไปข้างบนได้ผลลัพธ์ดีกว่าลงข้างล่าง และเมื่อมาถึงจุดยอดแล้ว จะไม่ลงไปไหนอีก เพราะไม่ว่าจะลงไปทางซ้ายหรือทางขวา คำตอบที่ได้จะแย่กว่าตำแหน่งที่อยู่ โดยเมื่อถึงจุดยอดแล้วไปไหนมาไหนไม่ได้แล้ว ก็จะหยุดค้นหาแล้วนำค่านั้นเป็นคำตอบ จากรูป **มีภูเขาแค่ลูกเดียว**

Greedy Algorithm



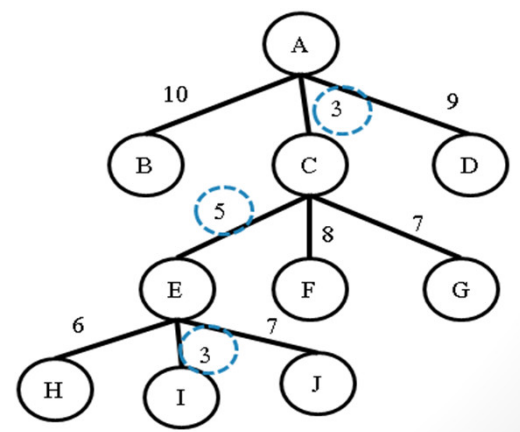
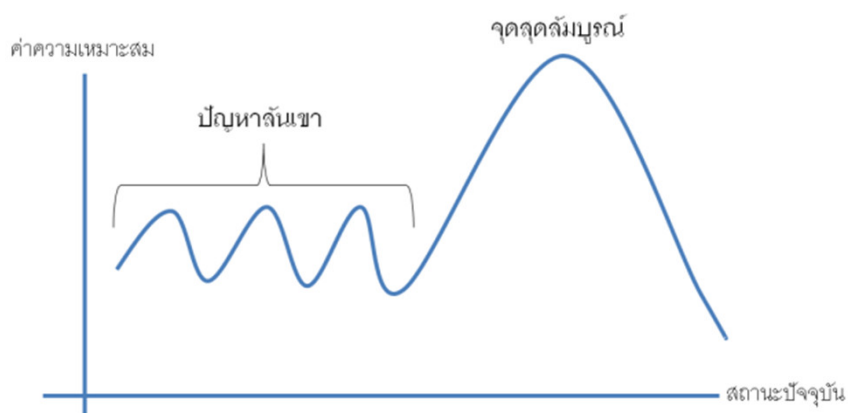
แต่ปัญหาการหาค่าที่เหมาะสมที่สุดส่วนมากเป็นรูปแบบภูเขาหลายยอด ดังรูป การค้นหาด้วยเทคนิคเชิงละโมภ เริ่มต้นที่วงกลมดำ ณ ตำแหน่งนี้ตัวค้นหาจะเคลื่อนที่ขึ้นไปทางด้านบน เมื่อถึงจุดยอดจะอยู่ตำแหน่งอยู่บนภูเขายอดต่ำหรือจุดสูงสุดระดับท้องถิ่นหรือจุดสุดสัมพัทธ์ (Local Optimum) สำหรับจุดสุดสัมพัทธ์ คือ จุดบนค่าของฟังก์ชันซึ่งเป็นค่าสูงสุดหรือต่ำสุด (จุดสูงสุดสัมพัทธ์หรือจุดต่ำสุดสัมพัทธ์) ในบริเวณใดบริเวณหนึ่งของโดเมน สำหรับค่านี้อาจจะมีได้ไม่จำกัดต่อหนึ่งฟังก์ชัน แต่ถ้าการค้นหาด้วยเทคนิคเชิงละโมภเริ่มต้นที่วงกลมม่วง ณ ตำแหน่งนี้ตัวค้นหาจะเคลื่อนที่ขึ้นไปทางด้านบน เมื่อถึงจุดยอดจะอยู่ตำแหน่งอยู่บนภูเขายอดที่สูงที่สุดเป็นจุดสูงสุดทั้งหมดหรือจุดสุดสัมบูรณ์ (Global Optimum) เป็นคำตอบที่ดีที่สุด สำหรับจุดสุดสัมบูรณ์ คือ จุดบนค่าของฟังก์ชัน ซึ่งเป็นค่าสูงสุดหรือต่ำสุด (จุดสูงสุดสัมบูรณ์หรือจุดต่ำสุดสัมบูรณ์) ตลอดโดเมนของฟังก์ชัน สำหรับค่านี้จะมีได้ไม่เกินหนึ่งค่าต่อหนึ่งฟังก์ชัน

Greedy Algorithm



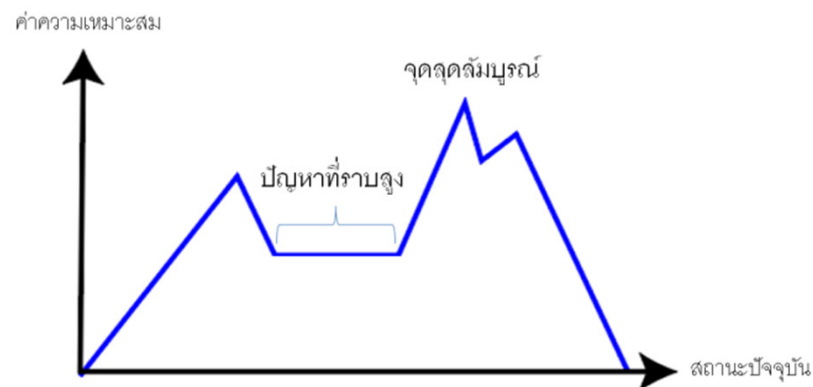
ปัญหาการหาค่าที่เหมาะสมที่สุดเฉพาะที่ (Local Optimum Problem) เป็นปัญหาที่เกิดจากการตัวค้นหาเริ่มต้นค้นหาผิดตำแหน่ง จากที่ได้กล่าวไปก่อนหน้านี้ รูปที่ 11.2 เมื่อตัวค้นหาอยู่ในตำแหน่งยอดเขาแล้ว จะไม่สามารถหาคำตอบที่ดีกว่าเดิมได้ จากเทคนิคเชิงละโมภที่จะไม่อนุญาตให้ตัวค้นหาเริ่มต้นเคลื่อนที่ไปตำแหน่งที่แย่ลงกว่าเดิม ทำให้เทคนิคเชิงละโมภจะไม่มีทางลงจากเขา เมื่อถึงยอดเขาแล้ว จึงไม่พบจุดสูงสุดทั้งหมด สำหรับวิธีการแก้ไขอาจจะใช้การสุ่มเริ่มต้นตำแหน่งอื่นแทนที่จะเริ่มต้นตำแหน่งเดิมซ้ำๆ ตัวอย่างถ้าต้องการแก้ปัญหาการเดินทางของพนักงานขาย เมื่อพิจารณาโนดที่แตกออกจากต้นไม้มัน ดังรูปที่ 11.3 สมมุติโนด A แตกออกมาได้ 2 ทาง จากต้นไม้มันแสดงให้เห็นว่า ถ้าใช้เทคนิคเชิงละโมภจะเลือกไปเส้นทางโนด B เพราะมีระยะทางน้อยกว่าคือ 5 เมื่อแตกกิ่งออกจะได้โนด D กับ E มีระยะทางเท่ากับ 10 กับ 11 ตามลำดับ ซึ่งแบบนี้คำตอบที่ได้ คือ 5 เพราะไม่พบคำตอบที่ดีกว่าเดิมแล้วจึงหยุดค้นหา แต่จริงคำตอบที่ดีที่สุด คือ 3 หรือ ไปแตกปม C ได้โนดลูกเป็น F กับ G มีระยะทางเท่ากับ 9 กับ 3 ซึ่งคำตอบอยู่ทางด้านปม G ถ้าค้นหาคำตอบทั้งหมดก็จะมีโอกาสพบคำตอบที่ดีที่สุด

Greedy Algorithm



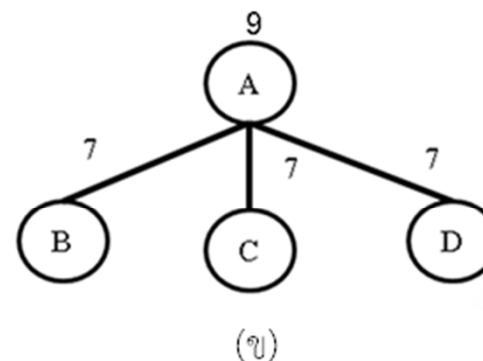
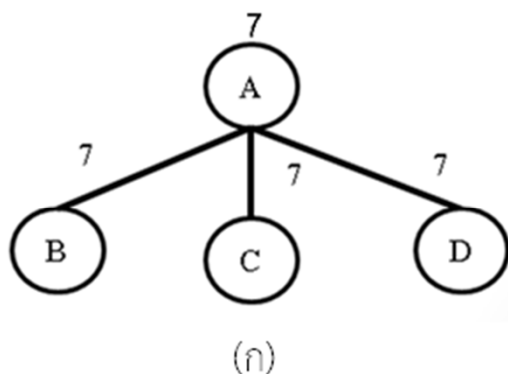
2) ปัญหาสันเขา (Ridge Problem) เป็นปัญหาการติดในจุดสูงสุดสัมพัทธ์แบบเป็นบริเวณใดบริเวณหนึ่ง (zone) การค้นหาเดินไปในทางที่ดีที่สุดตลอดเวลา แต่เมื่อเลือกเส้นทางที่ดีที่สุดไปเรื่อยๆ กลับกลายเป็นแอ่งในภายหลัง และจะกลับมาดีขึ้นเป็นช่วงบริเวณใดบริเวณหนึ่ง ถ้าเลือกใช้เทคนิคเชิงละโมภโปรแกรมจะหยุดก่อนจะออกจากบริเวณเหล่านี้ ดังรูปที่ 11.9 แสดงบริเวณที่เกิดปัญหาสันเขา ซึ่งตัวค้นหาถ้าอยู่ในบริเวณนี้และดำเนินการค้นหาด้วยขั้นตอนวิธีเทคนิคเชิงละโมภจะหยุดการค้นหาและติดอยู่ในจุดสูงสุดสัมพัทธ์ สำหรับวิธีการแก้ไขปัญหานี้อาจใช้วิธีเดียวกับการแก้ปัญหาการติดในจุดสูงสุดสัมพัทธ์ คือการเริ่มต้นค้นหาในตำแหน่งใหม่ๆ ตัวอย่างถ้าต้องการแก้ปัญหาการเดินทางของพนักงานขาย เมื่อพิจารณาโนดที่แตกออกจากต้นไม้ ดังรูปที่ 11.10 เมื่อเริ่มจากโนด A จะเลือกโนด C ซึ่งมีค่าดีที่สุดแล้ว ปรากฏ ได้โนด E F G ออกมา ตามหลักของเทคนิคเชิงละโมภต้องหยุดทำงานแล้ว แต่ถ้าทำงานต่อไป จะเลือกโนด E คำตอบที่ได้จากโนด E คือ โนด I ซึ่งดีกว่าโนด E ผลลัพธ์จะเป็นเช่นนี้เรื่อยๆ จนกว่าจะออกจากบริเวณที่เกิดปัญหาสันเขาจึงจะได้คำตอบที่ดีขึ้นและมีโอกาสจะพบจุดสูงสุดสัมบูรณ์ แต่ถ้าใช้ขั้นตอนวิธีเทคนิคเชิงละโมภ จะหยุดดำเนินการก่อนที่จะออกจากบริเวณจุดสูงสุดสัมพัทธ์

Greedy Algorithm



3) ปัญหาที่ราบสูง (Plateau Problem หรือ Flat Problem) คือ ปัญหาการติดในจุดสูงสุดสัมพัทธ์ที่เกิดจากตัวค้นหาอยู่บริเวณพื้นราบ คือคำตอบบริเวณโดยรอบตัวค้นหานี้จะเท่ากับคำตอบของตัวค้นหา หรือเป็นพื้นราบ ดังรูปที่ 11.11 หากการค้นหาอยู่ตำแหน่งพื้นราบ การที่จะเคลื่อนที่ไปด้านซ้ายหรือขวาก็ได้ผลลัพธ์เท่ากัน ในกรณีเทคนิคเชิงละโมภการค้นหาจะยุติการค้นหาคำตอบทันทีเมื่อติดอยู่ในบริเวณนี้ สำหรับวิธีการแก้ไขอาจจะเลือกทิศทางการเดินทางใดด้านหนึ่งเรื่อยๆ จนกว่าจะหลุดจากบริเวณพื้นราบแล้วสามารถค้นหาต่อไปได้ หรือใช้วิธีเดียวกับการแก้ปัญหาการติดในจุดสูงสุดสัมพัทธ์ คือการเริ่มต้นค้นหาในตำแหน่งใหม่ๆ

Greedy Algorithm



รูปที่ 11.12 ต้นไม้แสดงปัญหาที่ราบสูง

จากรูปที่ 11.12 ถ้าต้องการแก้ปัญหาการเดินทางของพนักงานขาย เมื่อพิจารณาโนดที่แตกออกจากต้นไม้ สามารถเกิดขึ้นได้ 2 กรณี กรณีแรกคือแตกออกมาแล้วคำตอบที่ได้ทั้งหมดเท่ากับ โหนดพ่อ กรณีจะยุติการค้นหาคำตอบทันที ดังรูปที่ 11.12 (ก) โหนดพ่อหรือโนด A มีขนาดเท่ากับ 7 เมื่อแตกออกมาจะมีโนดลูก 3 โหนดขนาดเท่ากับ 7 เหมือนกันหมด ไม่พบคำตอบที่ดีขึ้น จึงยุติการค้นหา กรณีที่สองคือแตกออกมาแล้วคำตอบที่ได้ทั้งหมดดีขึ้นกว่าโนดพ่อแม่ แต่คำตอบที่ได้ทั้งหมดเท่ากันหมด ดังรูปที่ 11.12 (ข) โหนดพ่อหรือโนด A มีขนาดเท่ากับ 9 เมื่อแตกออกมาจะมีโนดลูก 3 โหนดขนาดเท่ากับ 7 กรณีนี้จะไม่สามารถเลือกเส้นทางไหนได้ ด้วยเทคนิคเชิงละโมบนั้น เพราะโนดลูกทั้ง 3 ตัวที่แตกออกมานั้น มีระยะทางเท่ากับ 7 ทั้งหมด จะยุติการ ค้นหาแล้วนำ 7 เป็นคำตอบของปัญหา เพราะไม่สามารถหาเส้นทางที่จะค้นหาต่อไปได้อีก หรือไม่ก็ต้องมีขั้นตอนการจัดการในกรณีนี้ เช่น ถ้าเกิดกรณีเท่ากันให้เลือกเส้นทางแรกปม B ไปเลย แล้วค้นหาต่อไปซึ่งการทำงานเช่นนี้ นั้นไม่เหมาะสมและก็จะขัดกับหลักการทำงานของเทคนิคเชิงละโมบ แต่เนื่องจากไม่สามารถเลือกตัวใดเป็นคำตอบได้จึงต้องกำหนดแบบบังคับซึ่งการทำ แบบนี้อาจจะทำให้ได้ผลลัพธ์ที่ไม่เหมาะสม

ปัญหาการแลกเหรียญ (Coin changing problem)

- แคชเชียร์ 7-11 มีเหรียญ 1 บาท 2 บาท 5 บาท และ 10 บาท จำนวนไม่จำกัด ต้องการทอนเงิน 57 บาท อยากทราบว่าจะต้องใช้เหรียญเหล่านี้ในการทอนเงินให้พอดีน้อยที่สุดเท่าไร





เกณฑ์การเลือกของ Greedy

1. ทอนเหรียญที่มีมูลค่ามากที่สุดก่อนให้มากที่สุดเท่าที่จะเป็นไปได้
2. หากเงินมีค่าน้อยกว่ามูลค่าเหรียญแล้ว ให้ทอนเหรียญที่มีมูลค่ารองลงมา
3. ทำซ้ำเหมือนเดิมกับเหรียญที่มีมูลค่าน้อยลง จนกระทั่งเงินทอนมีค่าเป็นศูนย์หรือไม่สามารถทอนเงินได้อีกต่อไป



อัลกอริทึม Greedy กับ Coin changing

พัฒนาอัลกอริทึม greedy ในการทอนเหรียญให้น้อยที่สุด โดยใช้เกณฑ์ในการเลือกตามตัวอย่างที่ผ่านมา กำหนดให้อินพุตคือ อาเรย์ของมูลค่าเหรียญ $D[] = \{1, 5, 2, 10\}$ และ W คือจำนวนเงินที่ต้องการทอน

****ข้อสังเกต** สามารถใช้ได้มีประสิทธิภาพเนื่องจาก เหรียญต้องมีมูลค่ามากกว่า เหรียญเดิม 2 เท่าขึ้นไป วิธีนี้ถึงจะมีประสิทธิภาพ ตัวอย่างเช่น เหรียญ 5 25 30 35 40 แบบนี้วิธีนี้จะไม่ work เช่น 50 ถ้าใช้ 40 จะได้ 40 5 5 คือ 3 เหรียญ แต่ถ้าใช้ 25 จะแค่ 2 เหรียญ ดังนั้นวิธีนี้จึงไม่ work



ปัญหาเหมาะสำหรับอัลกอริทึมเชิงละโมบ

ตัวอย่าง

- ปัญหาถุงเป้ (knapsack)
- ปัญหาการเลือกกิจกรรม (Activity Selection Problem)
- ปัญหาการจัดตารางงาน (task scheduling)
- ปัญหาการจัดงาน (Assignment Problem หรือ Job Assignment)
- ปัญหาการบีบอัดข้อความ (text compression)

1. ปัญหา Knapsack

- เลือกของ K ชิ้นใส่กระเป๋าให้มีกำไร (profit) มากที่สุด โดยกระเป๋าไม่ขาด

Item	Weight	Profit
O1	18	25
O2	15	24
O3	10	15
O4	5	8



Weight limit
25



ปัญหา Knapsack

คำตอบของปัญหา 0/1 Knapsack สามารถแสดงในรูปของไบนารีเซตโดยที่ไอเท็มซึ่งถูกเลือก ('1') และไม่ถูกเลือก ('0')

Item	Weight	Profit
O1	18	25
O2	15	24
O3	10	15
O4	5	8

X	Profit	Weight
{0, 0, 1, 1}	$15 + 8 = 23$	$10 + 5 = 15$



ปัญหา Knapsack

สมมติจำนวนของ $N = 4$ และน้ำหนักกระเป๋าที่รับได้คือ $M = 25$ กำหนดให้ของแต่ละชั้นมีมูลค่า $(p_1, p_2, p_3, p_4) = (25, 24, 15, 8)$ และน้ำหนักคือ $(w_1, w_2, w_3, w_4) = (18, 15, 10, 5)$

1. เลือกของน้ำหนักน้อยที่สุดใส่ก่อน (Minimize weight)
2. เลือกของที่มีมูลค่ามากที่สุดใส่ก่อน (Maximize Profit)



ปัญหา Knapsack

สมมติ $n = 4$ และน้ำหนักถุงที่รับได้คือ $M=25$ กำหนดให้ของแต่ละชิ้นมีมูลค่า

$(p_1, p_2, p_3, p_4) = \{25, 24, 15, 8\}$ และน้ำหนักคือ $(w_1, w_2, w_3, w_4) = \{18, 15, 10, 5\}$

O1	O2	O3	O4
$25/18 = 1.39$	$24/15 = 1.6$	$15/10 = 1.5$	$8/5 = 1.6$

ใส่ของลงไปในถุงตามลำดับ : o2 -> o4 -> o3 -> o1

(x_1, x_2, x_3, x_4)	Profit	Weight
0 1 0 1	$24+8 = 32$	$15+5 = 20$

ปัญหา Knapsack

- เลือกสิ่งของโดยนำของซึ่งมีมูลค่ามากแต่น้ำหนักน้อยใส่ถุงก่อน
- **กำหนดให้ความหนาแน่น (density)** คือ สัดส่วนของมูลค่าและน้ำหนักของวัตถุ
ตัวอย่างเช่น วัตถุ A มีมูลค่า 25 น้ำหนัก 18 จะมีความหนาแน่นคือ $25/18 = 1.39$
- หลักการของ Greedy
 - เรียงลำดับของแต่ละชิ้นตาม**ความหนาแน่นของมันจากน้อยไปมาก**
 - สำหรับแต่ละขั้นตอน เลือกของที่มี**ความหนาแน่นมากใส่ลงไปก่อนจนกระทั่งไม่สามารถใส่ได้ (ถุงขาด)**



ปัญหา Knapsack

1. เรียงลำดับสิ่งของตามน้ำหนักที่น้อยที่สุดก่อน

o4->o3->o2->o1 $X = \{0, 0, 1, 1\}$ $P = 23$ $W = 10$

2. เรียงลำดับสิ่งของตามมูลค่าจากมากไปน้อย

o1->o2->o3->o4 $X = \{1, 0, 0, 1\}$ $P = 33$ $W = 23$

3. เรียงลำดับสิ่งของตามความหนาแน่นจากมากไปน้อย

o4->o2->o3->o1 $X = \{0, 1, 0, 1\}$ $P = 32$ $W = 20$

จากตัวอย่างนี้ ความหนาแน่น ไม่สามารถเป็นคำตอบที่ดีที่สุด

วิธีการเลือกตามความหนาแน่นของวัตถุที่ใช้ในปัญหา 0/1 Knapsack จะยังให้คำตอบที่เป็น Optimal หรือไม่? คือตอบคือไม่จากตัวอย่างนี้

ปัญหา 0/1 Knapsack อนุญาตให้ใส่ของลงไปในถุงทั้งวัตถุ ซึ่งมักจะทำให้ถุงเหือนำหนักบางส่วนทิ้งไว้ ตัวอย่างเช่น ในตัวอย่างที่แล้วเราจะเหือนำหนัก 5 หน่วยที่ไม่สามารถใส่ของได้แล้ว

Item	Weight	Profit	density
O1	18	25	1.39
O2	15	24	1.6
O3	10	15	1.5
O4	5	8	1.6

M หรือถุงมีความจุ = 25

ปัญหา Knapsack

- **ปัญหา Fractional knapsack** จะเพิ่มเงื่อนไขของปัญหาคือเราสามารถแบ่งของเพื่อนำไปใส่ในถุงได้ แต่มูลค่าของสิ่งนั้นจะลดลงตามสัดส่วนที่แบ่งด้วยเช่นกัน

	O1	O2	O3	O4
Profit	25	24	15	8
Weight	18	15	10	5
Density	1.39	1.6	1.5	1.6

(x1, x2, x3, x4)

1/4 0 1/2 1

Profit

$(25 \times 1/4) + (15 \times 1/2) + 8 = 21.75$

Weight

$(15 \times 1/4) + (5 \times 1/2) + 5 = 11.25$



ปัญหา Knapsack

O1	O2	O3	O4
$25/18 = 1.39$	$24/15 = 1.6$	$15/10 = 1.5$	$8/5 = 1.6$

ใส่ของลงไปในถุงตามลำดับ : o2 -> o4 -> **o3** -> o1

(x1, x2, x3, x4)

Profit

Weight

0 1 **1/2** 1

$$24 + (15 * \mathbf{1/2}) + 8 = \mathbf{39.5}$$

$$15 + 5 + (10 * \mathbf{1/2}) = \mathbf{25}$$

** ถ้ากรณีของ Fractional knapsack สามารถใช้ Greedy ความหนาแน่นได้
เพราะไม่มีช่องว่าง ดังนั้นมูลค่าสิ่งค่าจะขึ้นอยู่กับความหนาแน่นที่มากที่สุด



2. ปัญหาการเลือกกิจกรรม

ปัญหาการเลือกกิจกรรม (Activity Selection Problem) คือ มีห้องจัดกิจกรรมอยู่หนึ่งห้อง โดยห้องนี้มีผู้เข้ามาจัดกิจกรรมจำนวนมาก ภายในเวลาช่วงหนึ่ง ซึ่งกิจกรรมเหล่านี้มีโอกาสที่จะมีช่วงเวลาซ้อนทับกัน ถ้ากิจกรรมเกิดมีเวลาซ้อนทับกัน การจัดกิจกรรมจะไม่สามารถทำได้ต้องเลือกอย่างใดอย่างหนึ่ง เป้าหมายของปัญหานี้คือ ต้องการจัดกิจกรรมให้ได้มากที่สุด โดยที่ไม่มีกิจกรรมใดมีเวลาซ้อนทับกัน ตัวอย่างนี้คือมีห้องประชุมที่มีกิจกรรมดังตารางข้างล่างนี้

หมายเลขกิจกรรม	1	2	3	4	5
เวลาเริ่มต้น	1	4	5	2	2
เวลาสิ้นสุด	3	5	6	5	4



ปัญหาการเลือกกิจกรรม

หมายเลขกิจกรรม	1	2	3	4	5
เวลาเริ่มต้น	1	4	5	2	2
เวลาสิ้นสุด	3	5	6	5	4

กรณีใช้เทคนิคเชิงละโมภแก้ปัญหาการเลือกกิจกรรม ต้องกำหนดเกณฑ์ในการแก้ปัญหา ก่อน เช่น กรณีนี้ให้เลือกกิจกรรมที่เวลาสิ้นสุดน้อยที่สุด โดยเรียงลำดับกิจกรรมที่เวลาสิ้นสุดน้อยที่สุด ได้ลำดับดังนี้ 1, 5, 2, 4, 3 โดยกิจกรรมที่ 1 เวลาสิ้นสุด คือ 3 ซึ่งน้อยที่สุด และกิจกรรมที่ 5 เวลาสิ้นสุด คือ 4 ซึ่งน้อยที่สุดที่ลำดับถัดมา หลังจากเรียงลำดับกิจกรรมแล้ว ถ้าหากกิจกรรมใดที่มาภายหลังแล้วซ้อนทับกับกิจกรรมก่อนหน้าก็จะตัดกิจกรรมนั้นทิ้ง เมื่อพิจารณาการเสร็จสิ้นของกิจกรรมจะเรียงลำดับได้ดังนี้

เริ่มต้น กิจกรรมที่ 1 คำตอบที่ได้ คือ 1

ลำดับต่อมาคือ กิจกรรมที่ 5 ซึ่งเกิดการซ้อนทับกันของเวลาระหว่าง 1 กับ 5 ทำให้ไม่สามารถเลือกกิจกรรมที่ 5 ได้ คำตอบที่ได้ คือ 1

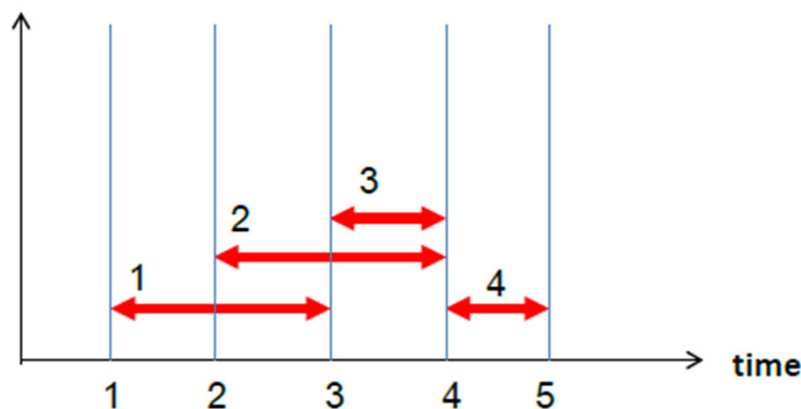
ลำดับต่อมาคือ กิจกรรมที่ 2 ซึ่งไม่เกิดการซ้อนทับกันของเวลา ทำให้เลือกกิจกรรมที่ 2 ได้ คำตอบที่ได้ คือ 1, 2

ลำดับต่อมาคือ กิจกรรมที่ 4 ซึ่งเกิดการซ้อนทับกันของเวลาระหว่าง 1, 2 กับ 4 ทำให้ไม่สามารถเลือกกิจกรรมที่ 4 ได้ คำตอบที่ได้ คือ 1, 2

ลำดับต่อมาคือ กิจกรรมที่ 3 ซึ่งไม่เกิดการซ้อนทับกันของเวลา ทำให้เลือกกิจกรรมที่ 3 ได้ คำตอบที่ได้ คือ 1, 2, 3 กิจกรรมหมดแล้ว ดังนั้นคำตอบของปัญหานี้ คือ 1, 2, 3

3. ปัญหาการจัดตารางงาน (Task Scheduling)

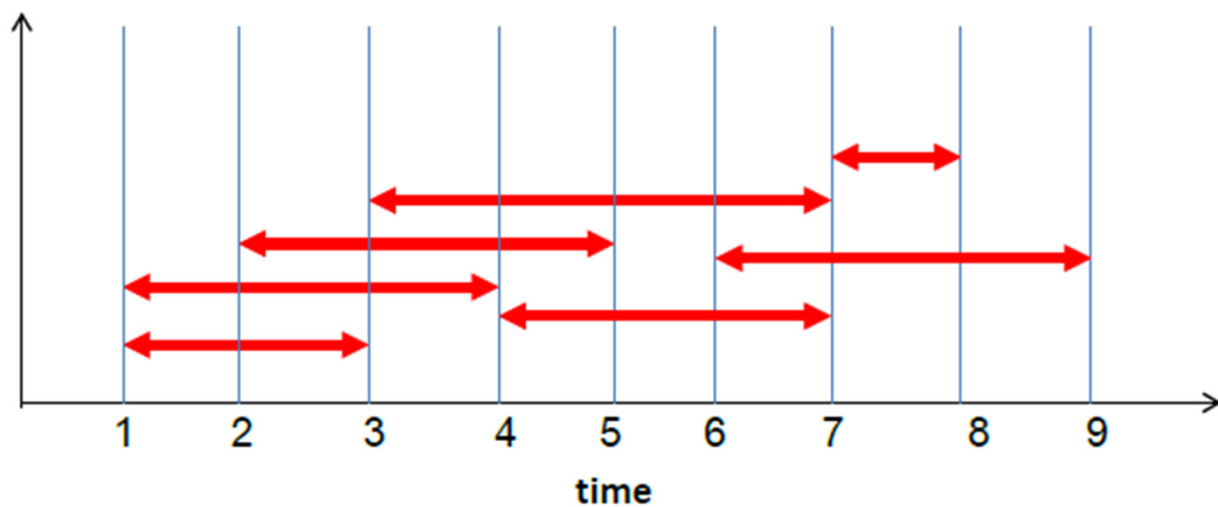
- กำหนดมีเซตของงานทั้งสิ้น n งาน ซึ่งรอที่จะถูกประมวลผลโดย machine แต่ละงานจะประกอบไปด้วย (s_i, f_i) ซึ่งหมายถึงเวลาเริ่มต้นและเวลาสิ้นสุดของการประมวลผลงานที่ i เนื่องจาก machine สามารถประมวลผลได้เพียงครั้งละ 1 งาน ดังนั้นหาก 2 งานมีเวลาซ้อนทับกัน (overlap time) machine จะต้องเลือก 1 งานมาประมวลผลและจะต้องยกเลิกอีกงานไป



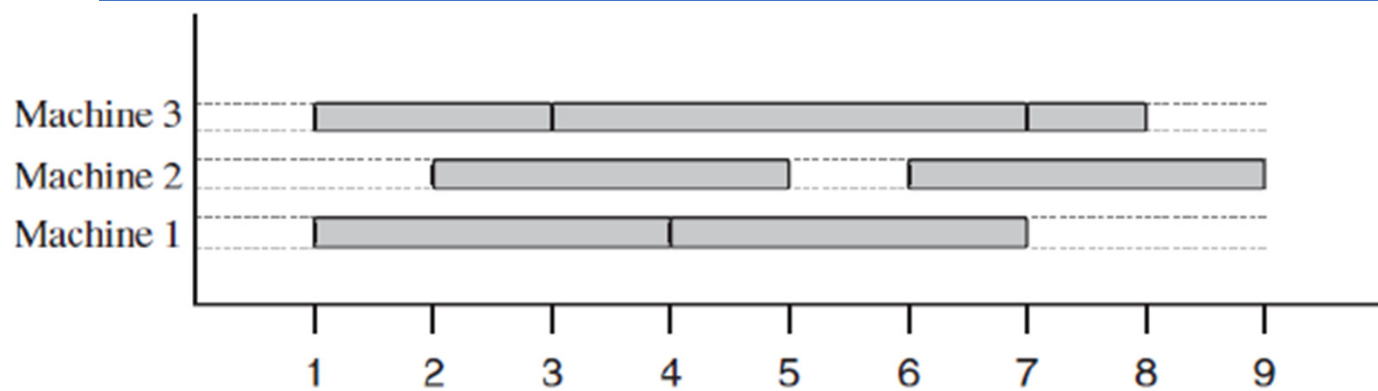
ประเด็นคือต้องการหาจำนวน machine ที่น้อยที่สุดเพื่อรองรับการประมวลผลงานทั้งหมด

หากเลือกประมวลผลงาน 2 จะต้องยกเลิกงาน 1 และงาน 3 หรือต้องการ machine เพิ่มอีก 1 ตัว เพื่อประมวลผลงาน 1 และงาน 3 ตามลำดับ

ตัวอย่าง



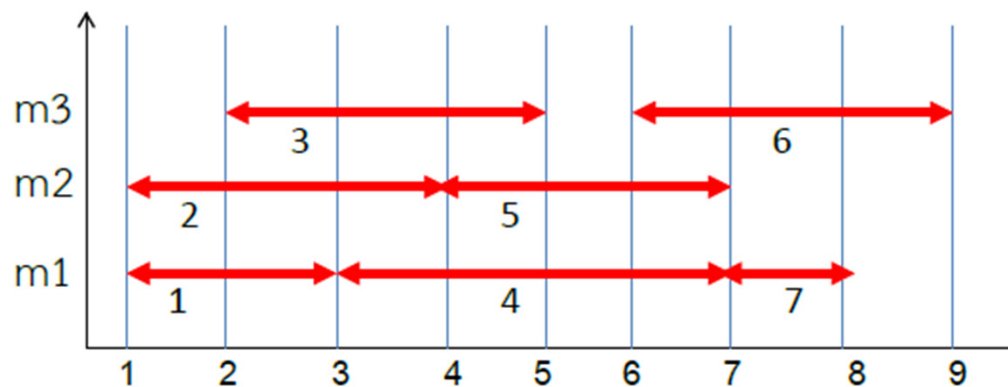
จะต้องใช้ machine อย่างน้อยที่สุด
3 ตัวในการประมวลผลงานทั้งหมด
โดยไม่ขัดแย้ง



Longest-first strategy

เลือกงานที่ใช้เวลาในการประมวลผลนานที่สุดก่อน

- เรียงลำดับงานตามเวลาที่ใช้ในการประมวลผลจากมากไปน้อย
- หากเกิดการทับซ้อนเวลาก็เพิ่ม machine อีก 1 ตัว



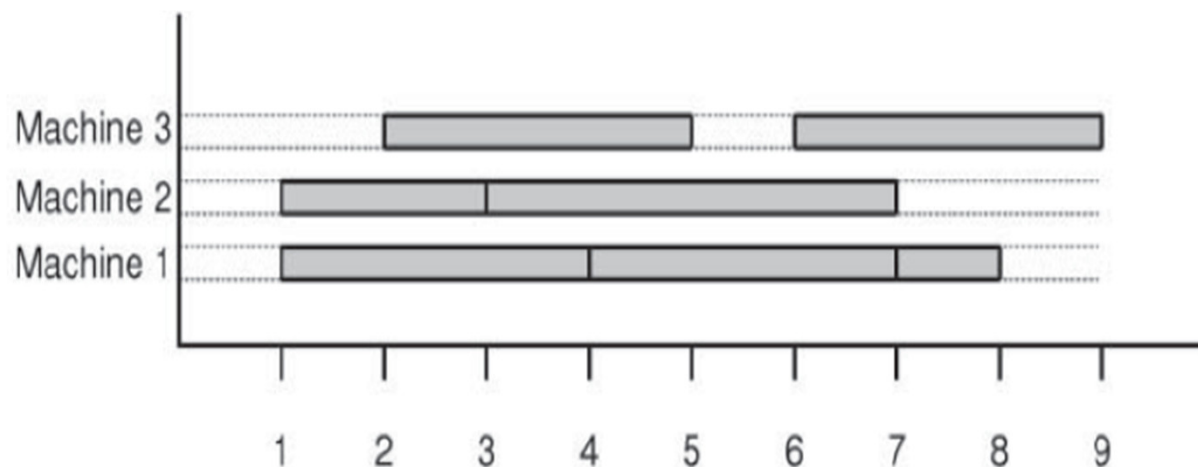
จะต้องใช้อย่างน้อย 3 machine

Task	s_i	f_i	rank
1	1	3	6
2	1	4	2
3	2	5	3
4	3	7	1
5	4	7	4
6	6	9	5
7	7	8	7

Earliest start-time strategy

เลือกงานที่มีเวลาเริ่มต้น (s_i) ก่อน

- เรียงลำดับงานตามเวลาเริ่มต้นจากน้อยไปมาก
- หากเกิดการทับซ้อนเวลาก็เพิ่ม machine อีก 1 ตัว



จะต้องใช้อย่างน้อย 3 machine

Task	s_i	f_i	rank
1	1	3	1
2	1	4	2
3	2	5	3
4	3	7	4
5	4	7	5
6	6	9	6
7	7	8	7



Other strategies for job scheduling

- Earliest finish-time strategy
เลือกงานที่มีเวลาสิ้นสุดก่อน
- Smallest interval strategy
เลือกงานที่ใช้เวลาประมวลผลน้อยที่สุดก่อน
- Least conflict strategy
เลือกงานที่มีช่วงเวลา conflict กับงานอื่นน้อยที่สุดก่อน 26



4. ปัญหาการจัดงาน (Assignment Problem หรือ Job Assignment)

ปัญหาการจัดงาน (Assignment Problem หรือ Job Assignment) คือ มอบหมายงาน n งานให้กับพนักงานทั้งหมด n คนทำ โดยให้เวลารวมในการทำงานน้อยที่สุดเท่าที่จะเป็นไปได้ สำหรับการแก้ปัญหานี้สามารถใช้เทคนิคเชิงละโมบ แก้ปัญหาโดยการจัดเรียงงานที่ใช้เวลาจากน้อยไปหามาก แล้วงานใดถูกเลือกแล้วให้กับพนักงานแล้ว ทั้งงานและพนักงานนั้นจะไม่สามารถถูกเลือกได้อีก ถ้าหากมีงานใดถูกเลือกอีกจะไม่นำมาพิจารณา เมื่องานทั้งหมดหรือพนักงานทั้งหมดถูกเลือกหมดแล้วก็จะหยุดดำเนินการต่อ เพื่อให้เกิดความเข้าใจของอธิบายตัวอย่างข้างล่างนี้

	งานที่ 0	งานที่ 1	งานที่ 2	งานที่ 3
คนที่ 0	9	2	7	8
คนที่ 1	6	4	3	7
คนที่ 2	5	8	1	8
คนที่ 3	7	6	9	4



ปัญหาการจัดงาน (Assignment Problem หรือ Job Assignment)

	งานที่ 0	งานที่ 1	งานที่ 2	งานที่ 3
คนที่ 0	9	2	7	8
คนที่ 1	6	4	3	7
คนที่ 2	5	8	1	8
คนที่ 3	7	6	9	4

จากตารางมีพนักงาน 4 คน มีงาน 4 งาน 1 คนทำงานได้ 1 งาน โดยตารางข้างล่างแสดงเวลาที่พนักงานแต่ละคนทำงานแต่ละงาน ต้องการจัดงานให้พนักงานใช้เวลาให้น้อยที่สุด
กรณีใช้เทคนิคเชิงลบไม่ต้องเลือกงานที่ใช้เวลาน้อยที่สุดในตาราง โดยเรียงลำดับงานดังนี้

จากตารางงานที่ใช้เวลาน้อยที่สุด คือ งานที่ 2 มอบให้คนที่ 2 ใช้เวลา 1 ชั่วโมง ดังนั้นจึงเลือกงานนี้เป็นงานแรก คำตอบ คือ งานที่ 2 กับ คนที่ 2

ต่อมางานที่ใช้เวลาน้อยเป็นลำดับต่อมา คือ งานที่ 1 มอบให้คนที่ 0 ใช้เวลา 2 ชั่วโมง ดังนั้นจึงเลือกงานนี้เป็นงานที่ 2 คำตอบ คือ งานที่ 2 กับ คนที่ 2 และ งานที่ 1 กับ คนที่ 0

ต่อมางานที่ใช้เวลาน้อยเป็นลำดับต่อมา คือ งานที่ 2 มอบให้คนที่ 1 ใช้เวลา 3 ชั่วโมง แต่งานที่ 2 ถูกเลือกไปแล้วจึงไม่พิจารณางานนี้

ต่อมางานที่ใช้เวลาน้อยเป็นลำดับต่อมา คือ งานที่ 1 มอบให้คนที่ 1 ใช้เวลา 4 ชั่วโมง แต่งานที่ 1 ถูกเลือกไปแล้วจึงไม่พิจารณางานนี้

ต่อมางานที่ใช้เวลาน้อยเป็นลำดับต่อมา คือ งานที่ 3 มอบให้คนที่ 3 ใช้เวลา 4 ชั่วโมง ดังนั้น จึงเลือกงานนี้เป็นงานที่ 3 คำตอบ คือ งานที่ 2 กับ คนที่ 2 และ งานที่ 1 กับ คนที่ 0 และ งานที่ 3 กับ คนที่ 3

ต่อมางานที่ใช้เวลาน้อยเป็นลำดับต่อมา คือ งานที่ 0 มอบให้คนที่ 5 ใช้เวลา 5 ชั่วโมง แต่งานที่ 0 ถูกเลือกไปแล้วจึงไม่พิจารณางานนี้

ต่อมางานที่ใช้เวลาน้อยเป็นลำดับต่อมา คือ งานที่ 0 มอบให้คนที่ 1 ใช้เวลา 6 ชั่วโมง ดังนั้นจึงเลือกงานนี้เป็นงานที่ 4 คำตอบ คือ งานที่ 2 กับ คนที่ 2 และ งานที่ 1 กับ คนที่ 0 และ งานที่ 3 กับ คนที่ 3 และ งานที่ 0 กับ คนที่ 1

เนื่องจากงานหรือคนนั้นหมดแล้ว จึงหยุดการมอบหมายงานต่อ แล้วได้คำตอบดังนี้ งานที่ 2 กับ คนที่ 2 และ งานที่ 1 กับ คนที่ 0 และ งานที่ 3 กับ คนที่ 3 และ งานที่ 0 กับ คนที่ 1

5. Text Compression

- ปกติแล้วคอมพิวเตอร์จัดเก็บไฟล์ในลักษณะที่เป็น fixed-length binary code ตัวอย่างเช่นสมมติไฟล์หนึ่งประกอบไปด้วยตัวอักษรและความถี่ที่ใช้งานดังนี้
- ถูกนำไปใช้งานจริงในแอปพลิเคชันสำหรับการบีบอัดไฟล์ PKZIP และการเข้ารหัสไฟล์ multimedia MP3

Character	Code	Frequency
a	0110 0001	10
e	0110 0101	15
i	0110 1001	12
s	0111 0011	3
t	0111 0100	4
r	0111 0010	1

TOTAL $45 * 8 = 360$ bits

ASCII Code

Decimal	Binary	Octal	Hex	ASCII	Decimal	Binary	Octal	Hex	ASCII	Decimal	Binary	Octal	Hex	ASCII	Decimal	Binary	Octal	Hex	ASCII
0	00000000	000	00	NUL	32	00100000	040	20	SP	64	01000000	100	40	@	96	01100000	140	60	`
1	00000001	001	01	SOH	33	00100001	041	21	!	65	01000001	101	41	A	97	01100001	141	61	a
2	00000010	002	02	STX	34	00100010	042	22	"	66	01000010	102	42	B	98	01100010	142	62	b
3	00000011	003	03	ETX	35	00100011	043	23	#	67	01000011	103	43	C	99	01100011	143	63	c
4	00000100	004	04	EOT	36	00100100	044	24	\$	68	01000100	104	44	D	100	01100100	144	64	d
5	00000101	005	05	ENQ	37	00100101	045	25	%	69	01000101	105	45	E	101	01100101	145	65	e
6	00000110	006	06	ACK	38	00100110	046	26	&	70	01000110	106	46	F	102	01100110	146	66	f
7	00000111	007	07	BEL	39	00100111	047	27	'	71	01000111	107	47	G	103	01100111	147	67	g
8	00001000	010	08	BS	40	00101000	050	28	(	72	01001000	110	48	H	104	01101000	150	68	h
9	00001001	011	09	HT	41	00101001	051	29	)	73	01001001	111	49	I	105	01101001	151	69	i
10	00001010	012	0A	LF	42	00101010	052	2A	*	74	01001010	112	4A	J	106	01101010	152	6A	j
11	00001011	013	0B	VT	43	00101011	053	2B	+	75	01001011	113	4B	K	107	01101011	153	6B	k
12	00001100	014	0C	FF	44	00101100	054	2C	,	76	01001100	114	4C	L	108	01101100	154	6C	l
13	00001101	015	0D	CR	45	00101101	055	2D	-	77	01001101	115	4D	M	109	01101101	155	6D	m
14	00001110	016	0E	SO	46	00101110	056	2E	.	78	01001110	116	4E	N	110	01101110	156	6E	n
15	00001111	017	0F	SI	47	00101111	057	2F	/	79	01001111	117	4F	O	111	01101111	157	6F	o
16	00010000	020	10	DLE	48	00110000	060	30	0	80	01010000	120	50	P	112	01110000	160	70	p
17	00010001	021	11	DC1	49	00110001	061	31	1	81	01010001	121	51	Q	113	01110001	161	71	q
18	00010010	022	12	DC2	50	00110010	062	32	2	82	01010010	122	52	R	114	01110010	162	72	r
19	00010011	023	13	DC3	51	00110011	063	33	3	83	01010011	123	53	S	115	01110011	163	73	s
20	00010100	024	14	DC4	52	00110100	064	34	4	84	01010100	124	54	T	116	01110100	164	74	t
21	00010101	025	15	NAK	53	00110101	065	35	5	85	01010101	125	55	U	117	01110101	165	75	u
22	00010110	026	16	SYN	54	00110110	066	36	6	86	01010110	126	56	V	118	01110110	166	76	v
23	00010111	027	17	ETB	55	00110111	067	37	7	87	01010111	127	57	W	119	01110111	167	77	w
24	00011000	030	18	CAN	56	00111000	070	38	8	88	01011000	130	58	X	120	01111000	170	78	x
25	00011001	031	19	EM	57	00111001	071	39	9	89	01011001	131	59	Y	121	01111001	171	79	y
26	00011010	032	1A	SUB	58	00111010	072	3A	:	90	01011010	132	5A	Z	122	01111010	172	7A	z
27	00011011	033	1B	ESC	59	00111011	073	3B	;	91	01011011	133	5B	[	123	01111011	173	7B	{
28	00011100	034	1C	FS	60	00111100	074	3C	<	92	01011100	134	5C	\	124	01111100	174	7C	
29	00011101	035	1D	GS	61	00111101	075	3D	=	93	01011101	135	5D	]	125	01111101	175	7D	}
30	00011110	036	1E	RS	62	00111110	076	3E	>	94	01011110	136	5E	^	126	01111110	176	7E	~
31	00011111	037	1F	US	63	00111111	077	3F	?	95	01011111	137	5F	_	127	01111111	177	7F	DEL

Data Types in C

Type	Size (bits)	Size (bytes)	Range
char	8	1	-128 to 127
unsigned char	8	1	0 to 255
int	16	2	-2^{15} to $2^{15}-1$
unsigned int	16	2	0 to $2^{16}-1$
short int	8	1	-128 to 127
unsigned short int	8	1	0 to 255
long int	32	4	-2^{31} to $2^{31}-1$
unsigned long int	32	4	0 to $2^{32}-1$
float	32	4	3.4E-38 to 3.4E+38
double	64	8	1.7E-308 to 1.7E+308
long double	80	10	3.4E-4932 to 1.1E+4932

Fixed length binary tree

จะเห็นได้ว่าค่อนข้างจะสิ้นเปลืองจำนวนบิตในการจัดเก็บมาก เนื่องจากมีตัวอักษรที่แตกต่างในไฟล์เพียง 6 ตัวเท่านั้น จึงเป็นไปได้ว่า เราสามารถลดจำนวนบิตลงให้สัมพันธ์กับความแตกต่างของตัวอักษรในไฟล์ได้

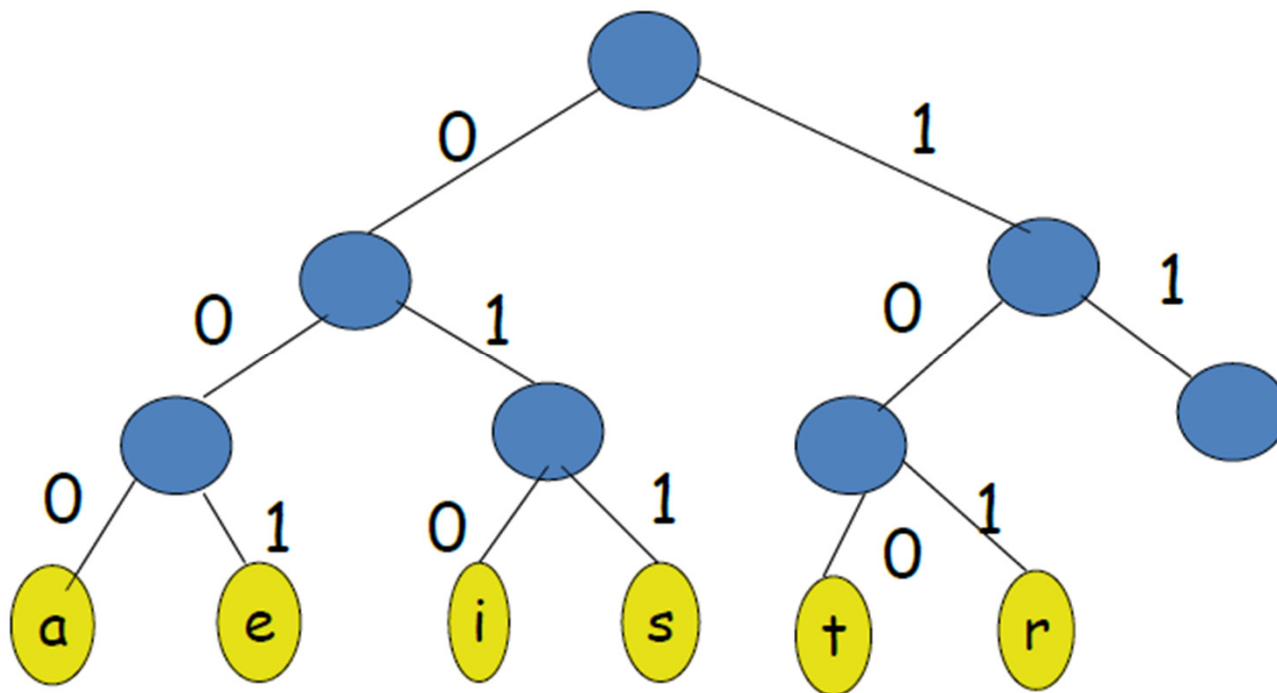
$$n \leq 2^{bit}$$

Character	Code	Frequency
a	000	10
e	001	15
i	010	12
s	011	3
t	100	4
r	101	1

TOTAL 45 * 3 = 135 bits

Fixed length binary tree

เราสามารถแสดงต้นไม้ของรหัสไบนารี เพื่อความรวดเร็วสำหรับการถอดรหัสดังนี้





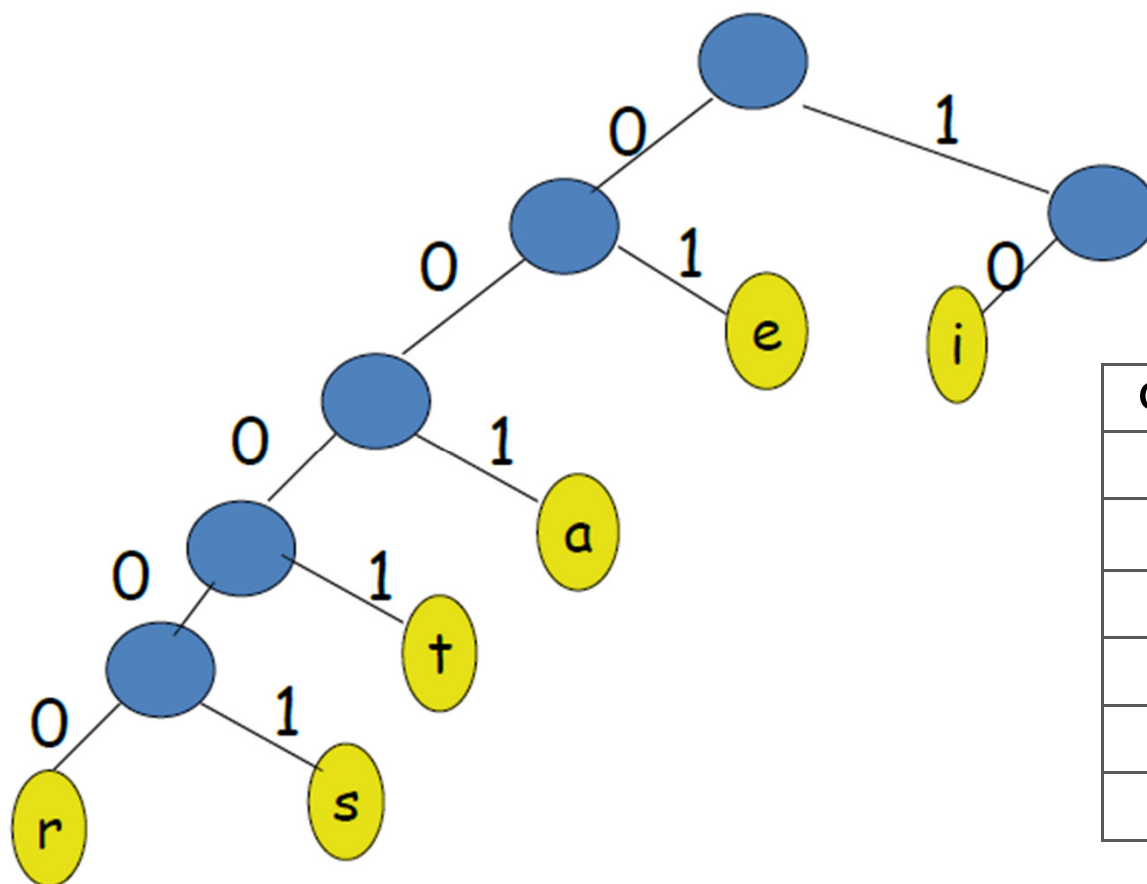
Variable-length binary coding

อย่างไรก็ตามถึงแม้ว่าจะลดพื้นที่ในการจัดเก็บได้ แต่ก็ยังคงไม่ optimal อยู่ดี ลองพิจารณาตัวอย่างด้านล่าง

Character	Code	Frequency
a	000	10
e	11	15
i	010	12
s	011	3
t	100	4
r	101	1

TOTAL $(30 \times 3) + (15 \times 2) = 90 + 30 = 120$ bits

Variable-length binary coding



Character	Code	Frequency
a	001	10
e	01	15
i	10	12
s	00001	3
t	0001	4
r	00000	1

TOTAL = 112 bits

Huffman Coding

Letter	Relative frequency in the English language
a	8.167%
b	1.492%
c	2.782%
d	4.253%
e	12.702%
f	2.228%
g	2.015%
h	6.094%
i	6.966%
j	0.153%
k	0.772%
l	4.025%
m	2.406%
n	6.749%
o	7.507%
p	1.929%
q	0.095%
r	5.987%
s	6.327%
t	9.056%
u	2.758%
v	0.978%
w	2.360%
x	0.150%
y	1.974%
z	0.074%

1. เริ่มจากนำทุกตัวอักษรและความถี่ หรือความน่าจะเป็น (probability) ก็ได้ ของมันมาสร้างเป็นไบนารี
2. หยิบไบนารี 2 ตัวที่มีความถี่ต่ำสุดมาเชื่อมกัน โดยสร้างเป็นต้นไม้หลัก
3. นำไบนารีที่มีความถี่น้อยสุดจากไบนารีที่เหลือมาเชื่อมกับต้นไม้ย่อยดังกล่าว แต่หากไบนารีที่เหลือมีค่าความถี่น้อยกว่าต้นไม้หลัก ให้รวมกับไบนารีอื่นเพื่อสร้างต้นไม้ย่อย จากนั้นจึงรวมต้นไม้ย่อยและต้นไม้หลักเข้าด้วยกัน
4. ทำเช่นนี้ไปเรื่อย ๆ จนกระทั่งต้นไม้ย่อยทั้งหมดรวมเป็นต้นไม้ต้นเดียว

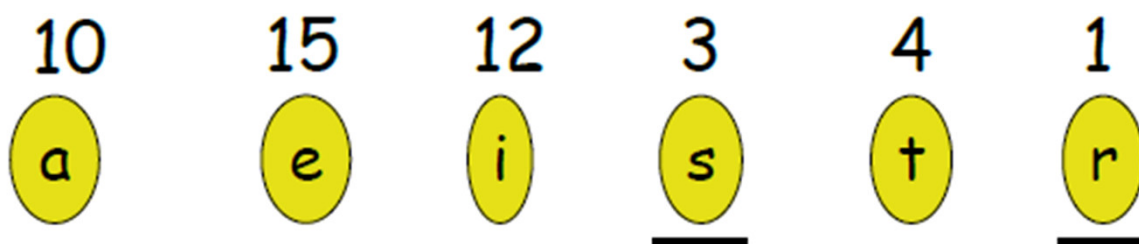


Huffman Coding

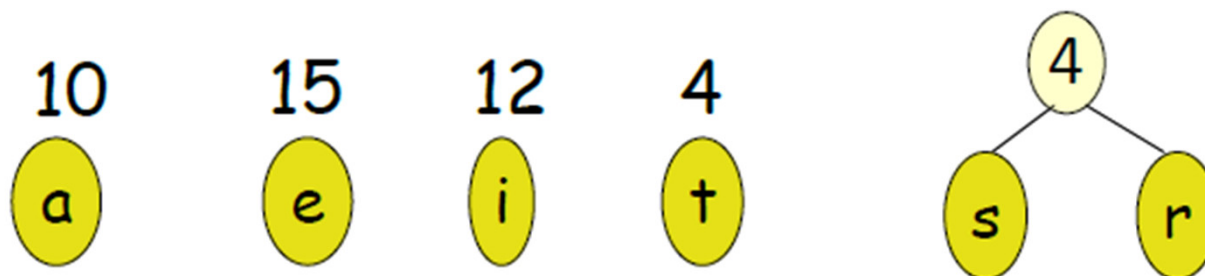
Huffman Coding เป็นเทคนิคในการบีบอัดข้อมูล เพื่อลดขนาดข้อมูลโดยไม่สูญเสียรายละเอียดใดๆ ได้รับการพัฒนาครั้งแรกโดย David Huffman โดยทั่วไปแล้วการเข้ารหัสฮัฟฟ์แมนจะมีประโยชน์ในการบีบอัดข้อมูลซึ่งมีอักขระเกิดขึ้นบ่อยครั้ง ถ้าเลือกตัวที่ซ้ำเยอะแต่บิตน้อยๆ และตัวที่ซ้ำน้อยๆ สามารถใช้บิตเยอะๆ ได้ จะสามารถลดความจุได้มาก โดยใช้หลักการของเทคนิคเชิงละโมบ โดยขั้นตอนการทำงานของ การเข้ารหัสฮัฟฟ์แมน มีรายละเอียดดังนี้

1. เริ่มต้นโดยการเรียงลำดับความถี่ของตัวอักษรจากมากไปหาน้อย นำตัวอักษรที่มีความถี่น้อยที่สุดมายุบรวมกัน
2. ทำการสร้างต้นไม้
 - 2.1 ถ้าตัวอักษรตัวต่อไปมีความถี่มากกว่าหรือเท่ากับตัวที่ยุบรวมกัน ก็เอามารวมกันต่อเป็นต้นไม้เดียวกัน ทำการยุบรวมกันเพื่อสร้างต้นไม้ไปเรื่อยๆ จนตัวอักษรทุกตัวปรากฏในต้นไม้
 - 2.2 ถ้าตัวอักษรต่อไปมีความถี่น้อยกว่าตัวที่ยุบรวมกัน ให้ข้ามไปพิจารณาตัวอักษรที่ถัดไปอีก แล้วพิจารณาตามขั้นตอนที่ 2.1 จนกว่าจะไม่พบตัวอักษรที่สามารถยุบรวมกันได้อีก ให้เริ่มต้นยุบรวมในส่วนของคุณ์อื่นๆ แทน
3. ดำเนินการใส่ตัวเลขบิต (0 กับ 1) เข้าไปเส้นเชื่อมระหว่างโนดของต้นไม้บนารีที่สร้างขึ้น ตัวเลขที่อยู่บนเส้นเชื่อมระหว่างโนด จากโนดรากถึงแต่ละตัวอักษรเป็นตัวแทนของการเข้ารหัส โดยตัวอักษรแต่ละตัวจะแทนโนดใบ

Example: Huffman Coding

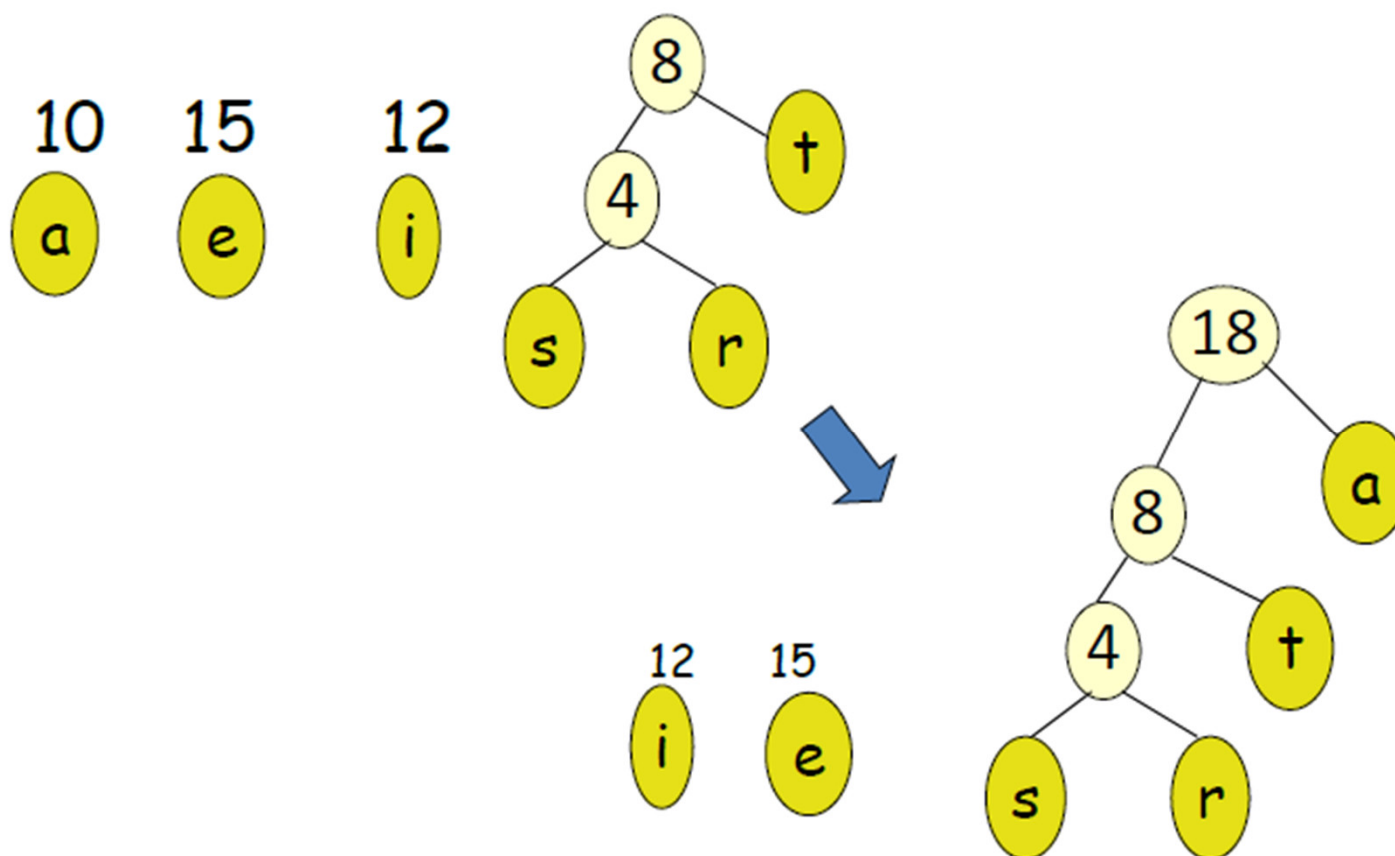


Character	Frequency
a	10
e	15
i	12
s	3
t	4
r	1

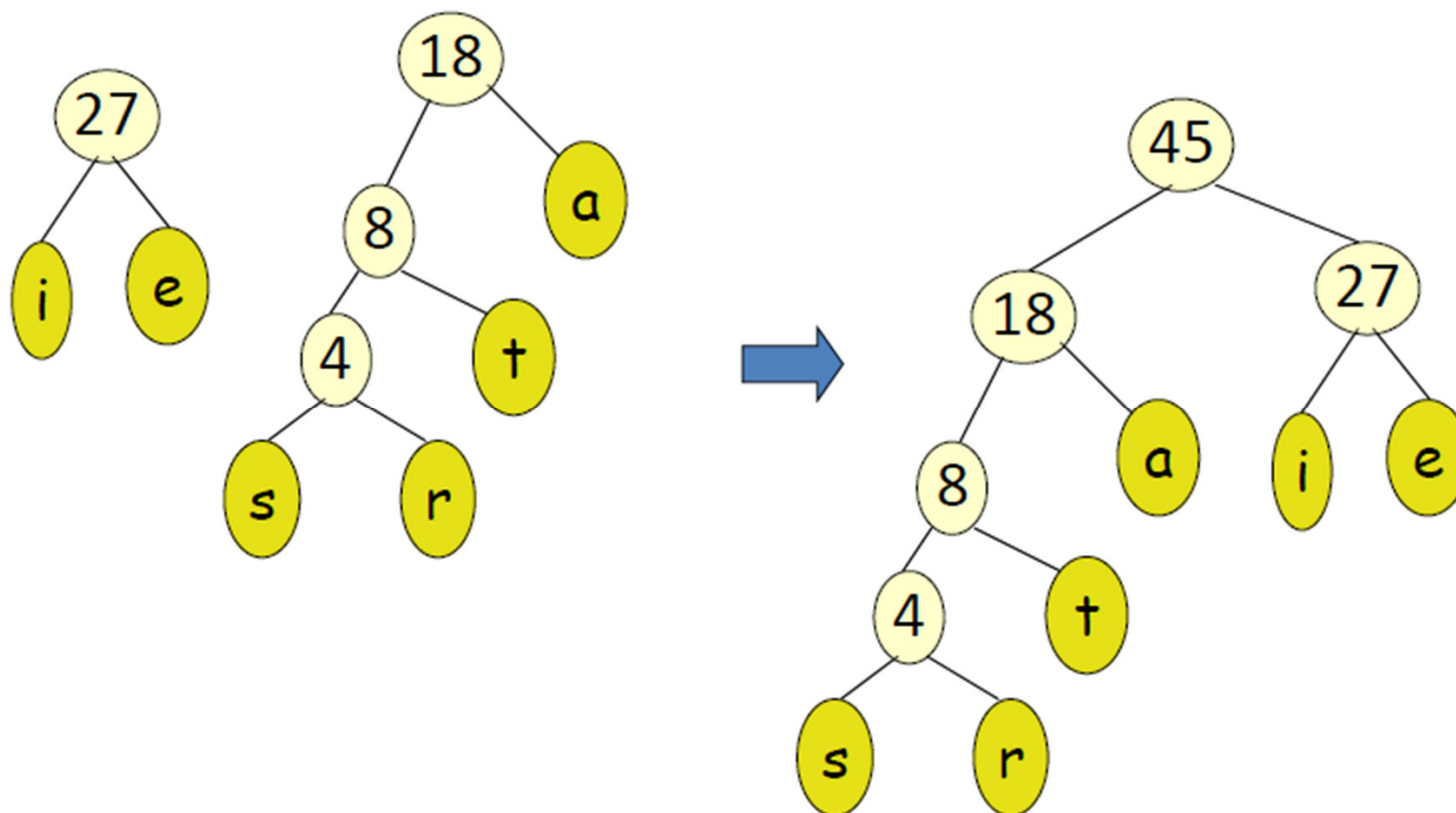


โหนดที่จะนำมารวมกับต้นไม้หลักต้องมีความถี่เท่ากันหรือมากกว่าต้นไม้หลัก

Example: Huffman Coding

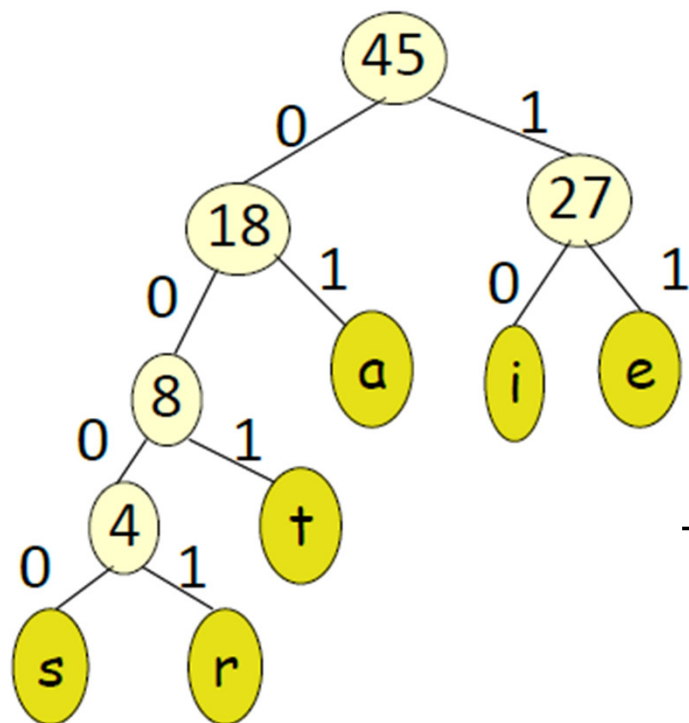


Example: Huffman Coding



Example: Huffman Coding

ให้กิ่งซ้ายมีค่าเป็น 0 และกิ่งขวามีค่าเป็น 1 จากนั้นรหัสของแต่ละตัวอักษรจะเริ่มไล่จาก root node



Character	Code	Frequency
a	01	10
e	11	15
i	10	12
s	0000	3
t	001	4
r	0001	1

$$\begin{aligned}
 \text{TOTAL} &= (10 \times 2) + (2 \times 15) + (2 \times 12) + (4 \times 3) + (3 \times 4) + (4 \times 1) \\
 &= 20 + 30 + 24 + 12 + 12 + 4 \\
 &= \mathbf{102 \text{ bits}}
 \end{aligned}$$



Example: Huffman Coding

ตัวอย่างที่ 2 ไฟล์หนึ่งมีตัวอักษรดังตารางข้างล่างนี้ จงใช้วิธีการเข้ารหัสฮัฟฟ์แมน เพื่อบีบอัดข้อมูลไฟล์นี้ และแสดงจำนวนบิตที่ต้องใช้ในการเก็บข้อมูลหลังจากการบีบอัด

ตัวอักษร	จำนวน
A	10
B	20
C	30
D	40
E	50
F	60

จากการคำนวณ ตัวอักษรทั้งหมด คือ 210 ตัวอักษรและแต่ละตัวใช้พื้นที่ในการจัดเก็บ 8 บิต เดิมไฟล์นี้ต้องใช้พื้นที่เก็บข้อมูล จำนวน 1,680 บิต



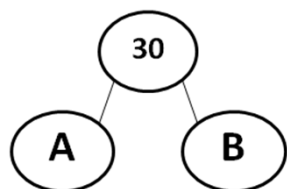
Example: Huffman Coding

ขั้นตอนที่ 1 เรียงลำดับความถี่ของตัวอักษรจากมากไปหาน้อย จากตัวอย่างนี้ ได้ผลลัพธ์ดังตารางข้างล่างนี้

ตัวอักษร	F	E	D	c	B	A
ความถี่	60	50	40	30	20	10

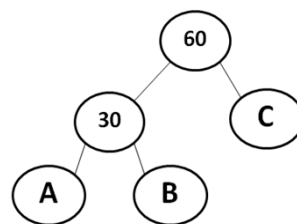
ขั้นตอนที่ 2 ทำการสร้างต้นไม้

2.1 เริ่มต้นจากตัวอักษรที่มีความถี่น้อยที่สุด คือ A กับ B ให้ดำเนินการยุบรวมกัน ได้ผลลัพธ์ดังรูป 12.8 โดยโนดพ่อแม่จะเป็นผลรวมความถี่ของตัวอักษร A กับ B ได้ความถี่รวมเท่ากับ 30

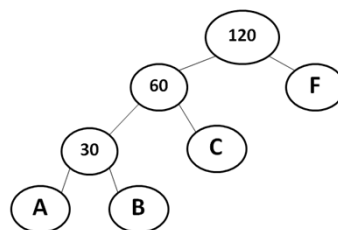


Example: Huffman Coding

2.2 ถ้าตัวอักษรตัวต่อไปมีความถี่มากกว่าหรือเท่ากับตัวที่ยุบรวมกัน ตัวอักษรที่ยุบรวม คือ A กับ B มีความถี่เท่ากับ 30 ตัวอักษรถัดไป คือ C ความถี่เท่ากับ 30 มีความถี่เท่ากัน จึงสามารถยุบรวมกันได้ ได้ผลลัพธ์ผลลัพธ์ผลลัพธ์ดังรูป 12.9 โดยโนดพ่อจะเป็นผลรวมความถี่ของตัวอักษร A กับ B กับ C ได้ความถี่รวมเท่ากับ 60

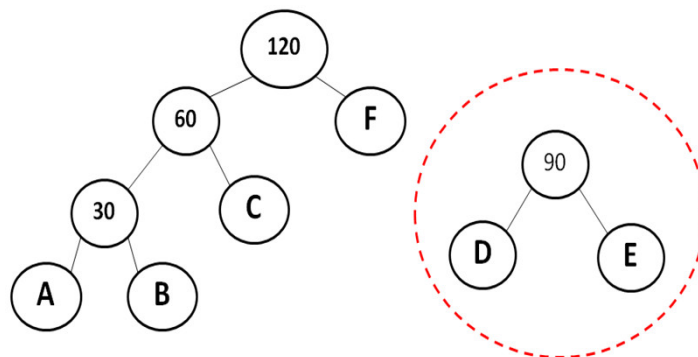


2.3 ถ้าตัวอักษรต่อไปมีความถี่น้อยกว่าตัวที่ยุบรวมกัน ให้ข้ามไปพิจารณาตัวอักษรที่ถัดไปอีก ตัวอักษรที่ยุบรวม คือ ตัวอักษร A กับ B กับ C มีความถี่เท่ากับ 60 ตัวอักษรถัดไปคือ D มีความถี่เท่ากับ 40 ซึ่งน้อยกว่า ความถี่ของตัวอักษรที่ยุบรวมกัน จึงเลื่อนไปพิจารณาตัวอักษรตัวถัดไป คือ E มีความถี่เท่ากับ 50 ซึ่งน้อยกว่า ความถี่ของตัวอักษรที่ยุบรวมกัน จึงเลื่อนไปพิจารณาตัวอักษรตัวถัดไป คือ F มีความถี่เท่ากับ 60 ซึ่งเท่ากับ ความถี่ของตัวอักษรที่ยุบรวมกัน จึงสามารถยุบรวมกันได้ ได้ผลลัพธ์ดังรูป 12.10 โดยโนดพ่อจะเป็นผลรวมความถี่ของตัวอักษร A กับ B กับ C กับ F ได้ความถี่รวมเท่ากับ 120



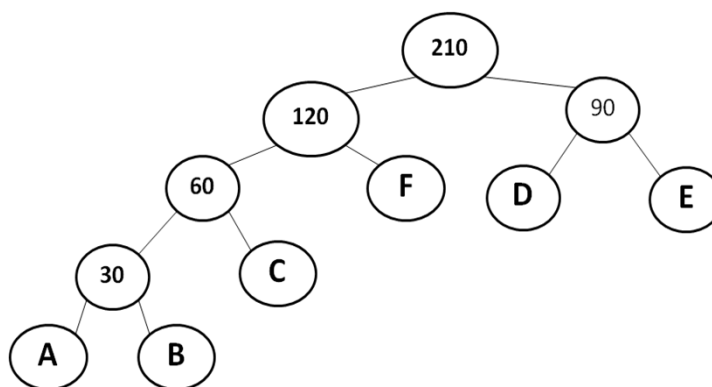
Example: Huffman Coding

2.4 เนื่องจากต้นไม้ที่ยุบรวมกัน คือ ตัวอักษร A กับ B กับ C กับ F มีความถี่มากกว่า ตัวอักษรถัดไปคือ D กับ E จึงให้ดำเนินการสร้างต้นไม้ใหม่ คือนำตัวอักษร D กับ E มารวมกันเป็นต้นไม้ใหม่ดังรูป 12.11 โดยโนดพ่อแม่จะเป็นผลรวมความถี่ของตัวอักษร D กับ E ได้ความถี่รวมเท่ากับ 90



Example: Huffman Coding

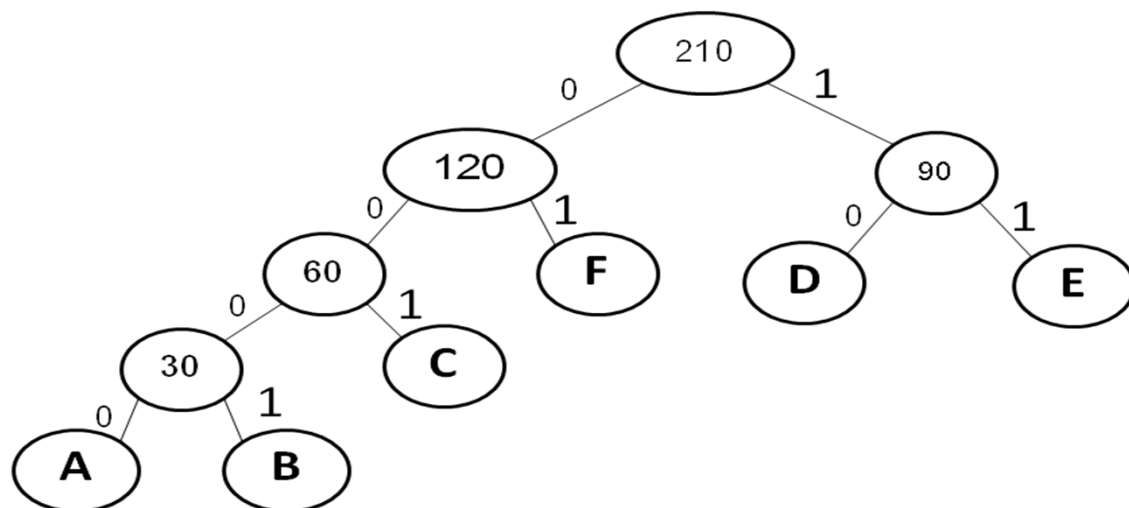
2.5 ถ้าตัวอักษรตัวต่อไปมีความถี่มากกว่าหรือเท่ากับตัวที่ยูบรวมกัน ตัวอักษรที่ยูบรวม คือ ตัวอักษร D กับ E มีความถี่เท่ากับ 90 ตัวอักษรถัดไป คือตัวอักษรที่ยูบรวม คือ A กับ B กับ C กับ F มีความถี่เท่ากับ 120 ซึ่งมีความถี่มากกว่า จึงสามารถยูบรวมกันได้ ได้ผลลัพธ์ดังรูปที่ 12.12 โดยโนดพ่อจะเป็นผลรวมความถี่ของตัวอักษร A กับ B กับ C กับ F กับ D กับ E ได้ความถี่รวมเท่ากับ 210 ตัวอักษรที่ใช้ในการสร้างต้นไม้หมดแล้ว ทำให้ขั้นตอนการสร้างต้นไม้เสร็จสิ้น



Example: Huffman Coding

ขั้นตอนที่ 3 ดำเนินการใส่ตัวเลขบิตเข้าไปเส้นเชื่อมระหว่างโนดของต้นไม้ไบนารีที่สร้างขึ้น ผลลัพธ์ได้ดังรูปที่ 12.13 จากต้นไม้สามารถสร้างตารางรหัสได้ดังตารางด้านขวามือ

การเข้ารหัสฮัฟฟ์แมน ผลลัพธ์ที่ได้จากการคำนวณ ได้ผลลัพธ์ดังนี้ ตัวอักษร A ใช้ 4 บิต ตัวอักษร B ใช้ 4 บิต ตัวอักษร C ใช้ 3 บิต ตัวอักษร D ใช้ 2 บิต ตัวอักษร E ใช้ 2 บิต ตัวอักษร F ใช้ 2 บิต ทำให้จำนวนบิตทั้งหมดที่ต้องใช้ คือ $10 \times 4 + 20 \times 4 + 30 \times 3 + 40 \times 2 + 50 \times 2 + 60 \times 2 = 510$ บิต สำหรับกรณีการเข้ารหัสความยาวบิตคงที่จำนวนบิตทั้งหมดที่ต้องใช้ คือ $210 \times 3 = 630$ บิต จะเห็นว่าหลักการเข้ารหัสฮัฟฟ์แมนสามารถลดพื้นที่ในการจัดเก็บบิตได้จำนวนมาก เมื่อเปรียบเทียบกับวิธีอื่นๆ



ตัวอักษร	รูปแบบบิต	จำนวน
A	0000	10
B	0001	20
C	001	30
D	10	40
E	11	50
F	01	60